Express.js: Building REST APIs and Optimizing Web Applications

Comprehensive Article / Assignment

Student Name: Rishikeshav Jha

Class: Computer Engineering / TE-A

Subject: Web / Backend Development

This article covers Express.js fundamentals, application setup, routing, middleware, REST API development, advanced techniques, security, testing, and performance optimization.

1. Introduction to Express.js

Express.js is a lightweight and flexible web framework for Node.js. It simplifies HTTP request handling, routing, middleware, and API development. Express is widely used for backend services because it is easy to learn and can scale from small applications to larger systems.

Express does not impose a single architecture. Developers can organize routes, controllers, models, middleware, and services according to project requirements.

2. Key Features of Express.js

• Simple HTTP routing
• Reusable middleware
• REST API support
• JSON request and response handling
• Centralized error handling
• Integration with databases and npm packages

3. Setting Up an Express Application

Install Node.js and npm, create a project, initialize npm, and install Express.

Example: Setup

```
mkdir express-demo
cd express-demo
npm init -y
npm install express
```

Example: Basic Server

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
  res.send('Hello from Express.js!');
});

app.listen(PORT, () => console.log(`Server running on ${PORT}`));
```

4. Routing in Express

Routing defines how the server responds to a URL and HTTP method. Express supports GET, POST, PUT, PATCH, and DELETE. Route parameters can identify specific resources.

Example: Routes

```
app.get('/users', (req, res) => {
  res.json({ message: 'List of users' });
});

app.get('/users/:id', (req, res) => {
  res.json({ userId: req.params.id });
});

app.post('/users', (req, res) => {
  res.status(201).json({ message: 'User created' });
});
```

5. Middleware

Middleware functions run during the request-response cycle. They can log requests, authenticate users, validate input, parse request data, or pass control with next(). Express provides built-in middleware such as express.json() and express.static().

Example: Custom Middleware

```
const logger = (req, res, next) => {
  console.log(`${req.method} ${req.originalUrl}`);
  next();
};

app.use(logger);
app.use(express.json());
```

6. Building REST APIs with Express

REST APIs expose resources through URLs and use HTTP methods to perform CRUD operations. GET reads data, POST creates data, PUT/PATCH updates data, and DELETE removes data. JSON is commonly used for API communication.

Example: CRUD REST API

```
const express = require('express');
const app = express();
app.use(express.json());

let products = [
  { id: 1, name: 'Keyboard', price: 900 },
  { id: 2, name: 'Mouse', price: 500 }
];

app.get('/api/products', (req, res) => res.json(products));

app.get('/api/products/:id', (req, res) => {
  const p = products.find(x => x.id === Number(req.params.id));
  if (!p) return res.status(404).json({ error: 'Product not found' });
  res.json(p);
});

app.post('/api/products', (req, res) => {
  const p = { id: products.length + 1, name: req.body.name, price: req.body.price };
  products.push(p);
  res.status(201).json(p);
});

app.put('/api/products/:id', (req, res) => {
  const p = products.find(x => x.id === Number(req.params.id));
  if (!p) return res.status(404).json({ error: 'Product not found' });
  p.name = req.body.name ?? p.name;
  p.price = req.body.price ?? p.price;
  res.json(p);
});

app.delete('/api/products/:id', (req, res) => {
  const i = products.findIndex(x => x.id === Number(req.params.id));
  if (i === -1) return res.status(404).json({ error: 'Product not found' });
  products.splice(i, 1);
  res.status(204).send();
});
```

7. Modular Routing

Large applications should separate routes from the main server file. Express Router helps group related endpoints and makes the code easier to maintain.

Example: Express Router

```
// routes/products.js
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => res.json({ message: 'All products' }));
router.get('/:id', (req, res) => res.json({ id: req.params.id }));

module.exports = router;

// server.js
const productRoutes = require('./routes/products');
app.use('/api/products', productRoutes);
```

8. Request Data

Route parameters are available through req.params, query-string values through req.query, and parsed JSON request bodies through req.body.

Example

```
app.get('/search', (req, res) => {
  res.json({ keyword: req.query.keyword });
});

app.get('/users/:id', (req, res) => {
  res.json({ id: req.params.id });
});
```

9. Error Handling

Centralized error-handling middleware keeps API responses consistent and avoids duplicated error logic. An Express error handler receives err, req, res, and next.

Example: Error Handler

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({
    error: err.message || 'Internal Server Error'
  });
});
```

10. Database Integration

Express does not provide its own database layer. It can work with MongoDB, PostgreSQL, MySQL, and other databases through suitable drivers or libraries. Separating database models or services from route handlers improves maintainability and testing.

11. Authentication and Security

Production APIs should use authentication and authorization where required. Important practices include password hashing, input validation, HTTPS, secure headers, appropriate CORS configuration, rate limiting, and keeping secrets in environment variables rather than source control.

Example: Environment Variables

```
// .env
PORT=3000
DATABASE_URL=your_database_connection_string

// application code
require('dotenv').config();
const port = process.env.PORT || 3000;
```

12. Advanced Express Techniques

Useful techniques for larger applications include modular routing, controllers and service layers, request validation, asynchronous error handling, configuration management, caching, logging, monitoring, compression, and rate limiting. These practices make applications easier to maintain and operate.

13. Performance Optimization

Use asynchronous I/O and avoid unnecessary synchronous work. Optimize database queries and add indexes for frequently searched fields. Return only required data, cache suitable repeated results, enable compression where useful, and monitor slow requests, errors, memory usage, and database bottlenecks. Load testing can help identify capacity limits before deployment.

Example: Compression

```
const compression = require('compression');
app.use(compression());
```

14. Testing Express APIs

Automated tests verify routes, status codes, validation, and response data. Mocha, Chai, and Supertest are commonly used together for Express API testing.

Example: API Test

```
const request = require('supertest');
const { expect } = require('chai');
const app = require('../app');

describe('GET /api/products', () => {
  it('returns products', async () => {
    const response = await request(app).get('/api/products').expect(200);
    expect(response.body).to.be.an('array');
  });
});
```

15. Recommended Project Structure

```
express-app/
■■■ app.js
■■■ server.js
■■■ routes/
■   ■■■ productRoutes.js
■■■ controllers/
■   ■■■ productController.js
■■■ models/
■   ■■■ productModel.js
■■■ middleware/
■   ■■■ errorHandler.js
■■■ services/
■   ■■■ productService.js
■■■ tests/
■   ■■■ product.test.js
■■■ .env
■■■ .gitignore
■■■ package.json
```

16. Conclusion

Express.js provides a practical foundation for Node.js web servers and REST APIs. Its routing and middleware model keeps request processing simple while allowing applications to grow into organized backend systems.

A reliable Express application should be maintainable, secure, testable, and efficient. Modular design, validation, error handling, database optimization, automated testing, logging, and monitoring help create production-ready applications.

17. References / Further Reading

Express.js documentation — expressjs.com
Node.js documentation — nodejs.org
MDN Web Docs — HTTP and web development
npm documentation — JavaScript package management